LEOFIT: SISTEMA WEB PWA DE GESTIÓN DE PEDIDOS

Sustentación Final — Arquitectura, Calidad, Interoperabilidad, Usabilidad y Cloud

Curso: Curso Integrador II: Software (100000S12F) — Ciclo 2026-I
Institución: Universidad Tecnológica del Perú (UTP)
Organización Beneficiaria: LeoFit Confecciones y Diseños Deportivos S.A.C.

Equipo de Ingeniería (Grupo 01):

Lady Luz Loayza Rodriguez — Scrum Master / DevSecOps
Víctor Leandro Cárdenas Fernández — Product Owner / DBA
Harley Anthony Roman Delgado — Frontend Lead / PWA
Jim Alessandro Dávila Morales — QA Lead / Backend
Daniel Enrique Rojas Sanchez — Analista de Negocio / Cloud DevOps

I. El Problema y la Solución

Contexto operativo de LeoFit en Gamarra

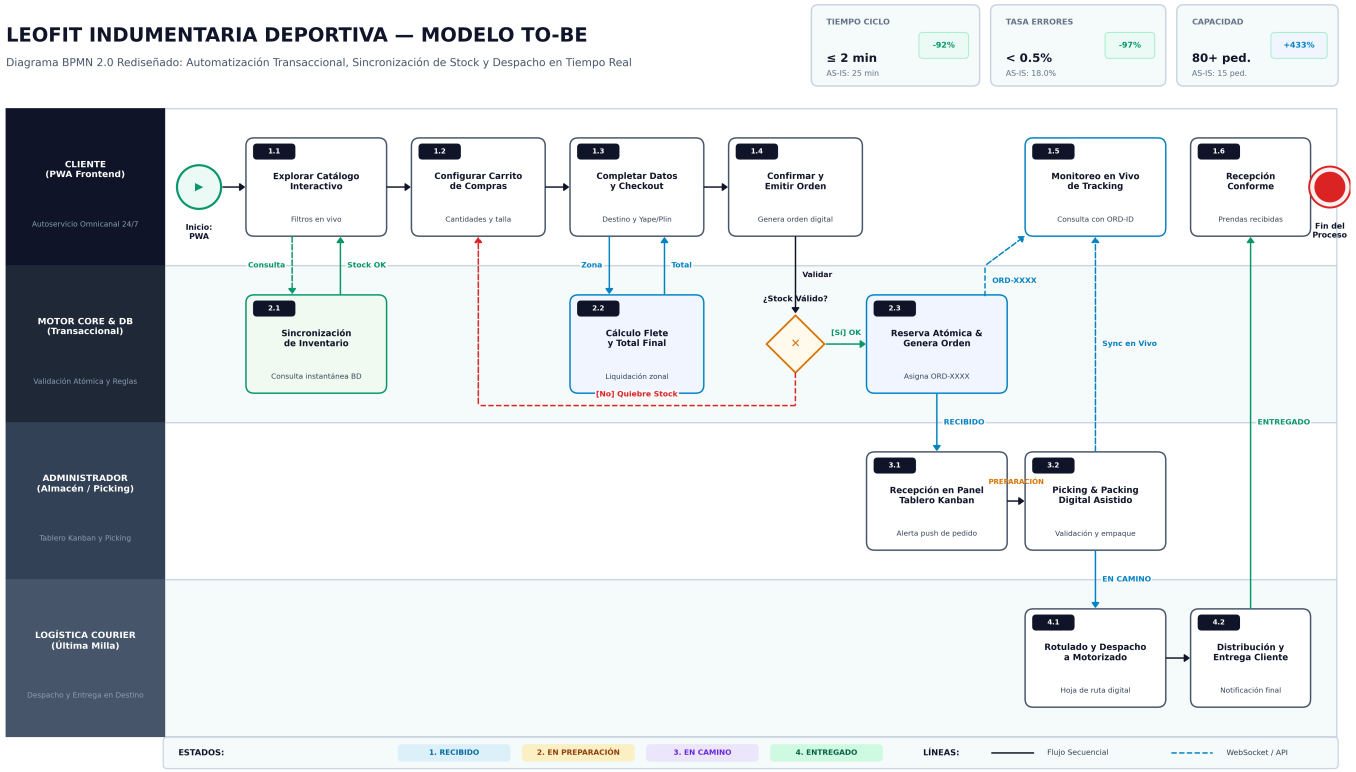
Expositor: Daniel Enrique Rojas Sanchez

De un Flujo Manual a un Sistema Transaccional

Dimensión	Situación Inicial (AS-IS)	Con LeoFit PWA
Canal de pedidos	Chats de WhatsApp no estructurados y libreta física	Catálogo PWA con carrito y pedido estructurado
Tiempo de atención	45 a 120 minutos por orden	38.2 segundos de captura de pedido
Error en despachos	14.2%	0.4% de errores operativos
Visibilidad de stock	Inexistente por talla y color	Stock por variante con descuento atómico
Intención de compra perdida	22% por falta de disponibilidad visible	Disponibilidad en tiempo real para el cliente

- **Empresa:** LeoFit Confecciones y Diseños Deportivos S.A.C. — Emporio Comercial de Gamarra, La Victoria, Lima.

Proceso Rediseñado (BPMN 2.0 TO-BE)

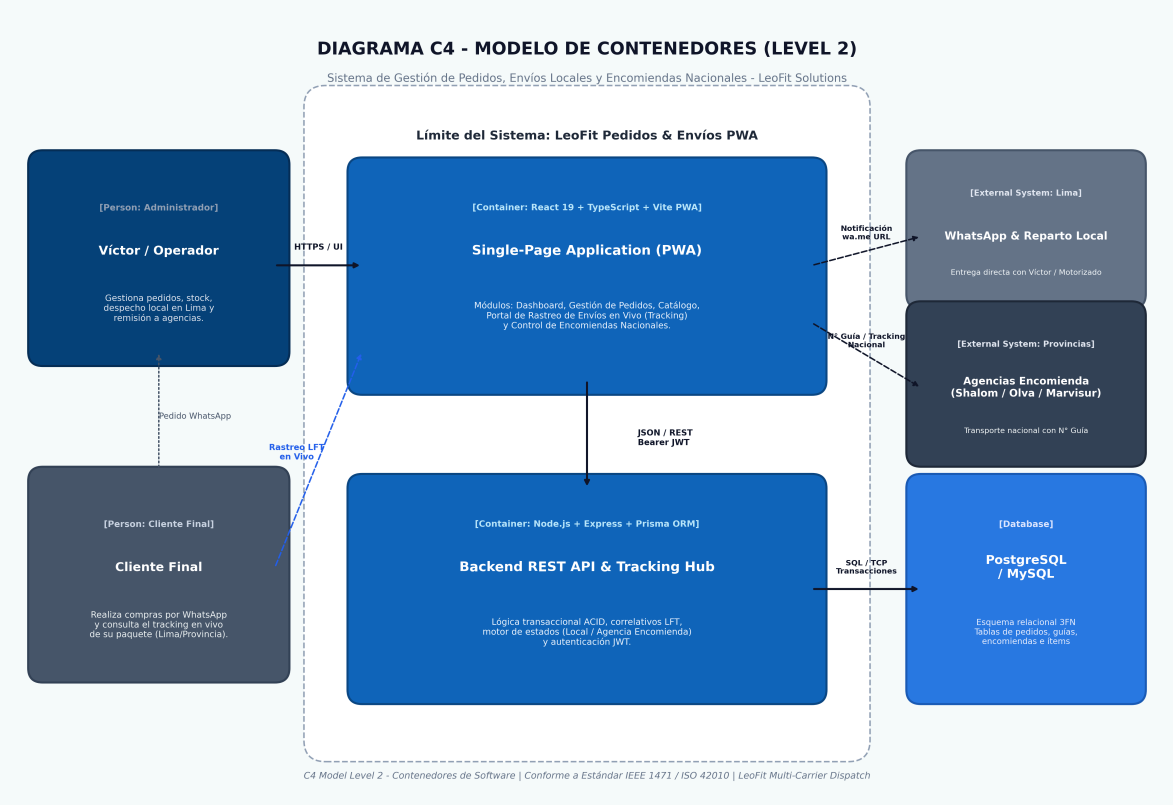


II. Arquitectura, Datos y Seguridad

Clean Architecture sobre PostgreSQL 16

Expositor: Víctor Leandro Cárdenas Fernández

Arquitectura de Contenedores (C4 — Nivel 2)

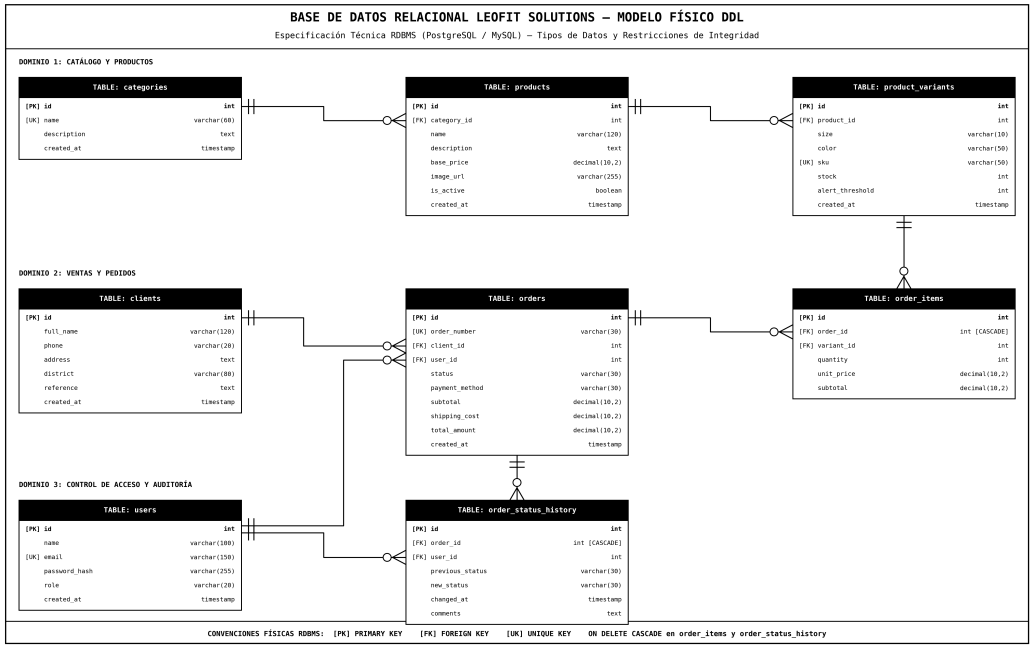


Stack Tecnológico

Capa	Tecnología	Decisión de Diseño
Frontend	React 19, TypeScript, Vite 6, Tailwind CSS 4	PWA reactiva, catálogo con validación de variantes y comprobantes con jsPDF
Backend	Node.js 20 LTS, Express 4.19, TypeScript 5.4	Contratos validados con Zod y respuesta uniforme `{success, data | error}`
Base de datos	PostgreSQL 16	12 tablas, 2 vistas, 14 índices B-Tree; normalización 3FN/BCNF
Calidad	Jest, Supertest, Vitest, Testing Library	101 pruebas automatizadas aprobadas
DevOps	GitHub Actions, GitHub Pages, Render	CI/CD automatizado y despliegue multi-proveedor

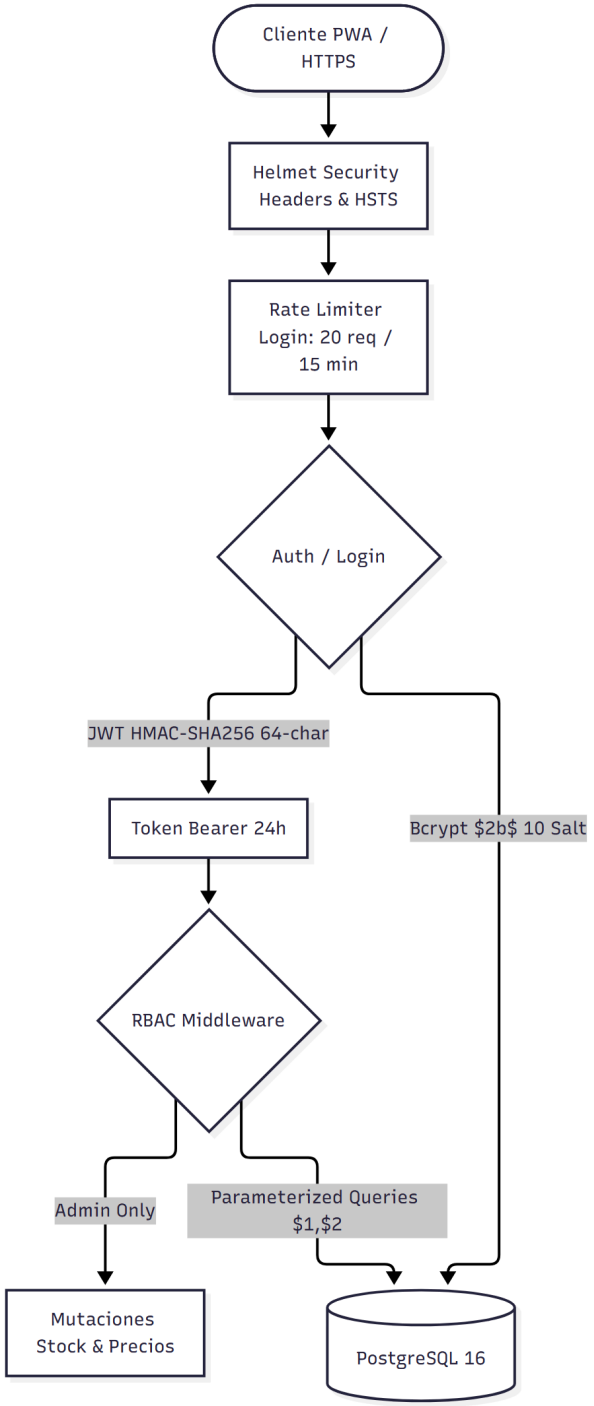
Modelo de Datos y Consistencia Transaccional

- **12 tablas, 2 vistas, 2 enums y 14 índices B-Tree**, normalizados en 3FN y BCNF.
- **Descuento atómico de stock** mediante el trigger `trg_decrement_stock_on_order`.
- **Concurrencia:** bloqueo pesimista `SELECT ... FOR UPDATE` con reintentos y retroceso exponencial ante deadlocks (códigos 40001 / 40P01).
- **Máquina de estados** del pedido: `RECIBIDO` → `PREPARACION` → `EN_CAMINO` → `ENTREGADO` (o cancelado).



Seguridad por Diseño

- **RBAC** con roles `ADMIN` y `OPERATOR`.
- **Contraseñas:** bcryptjs con 10 rondas y hash ficticio para mitigar *timing attacks*.
- **JWT HS256** con validación *fail-fast*: la API no arranca sin `JWT_SECRET`.
- **Webhooks de pago:** firma HMAC-SHA256 con `timingSafeEqual`.
- **Defensa en profundidad:** cabeceras Helmet, validación Zod y consultas parametrizadas.



Evolución desde el APF2

Aspecto	Estado en APF2	Resultado en esta Entrega
Interoperabilidad	Integraciones sin pruebas automatizadas	Suite `integration_external.test.ts` con 7 casos aprobados
Usabilidad	Sin evaluación formal	Pruebas `usability_iso25010.test.ts` (9) y estudio SUS con 10 usuarios: 88.5 / 100
Trazabilidad	Historias sin vínculo explícito a pruebas	Matriz HU-001 a HU-008 contra suites de Jest y Vitest
Despliegue	Despliegue inicial	Versión 2 en GitHub Pages y Render con 3 workflows de GitHub Actions

III. Calidad Funcional

Pruebas automatizadas y cobertura

Expositor: Jim Alessandro Dávila Morales

Suites de Prueba del Backend

Suite	Casos	Alcance
`orders.test.ts`	13	Creación transaccional, tracking, paginación y campos logísticos
`auth.test.ts`	8	Registro, login, hash bcrypt y JWT con RBAC
`integration_external.test.ts`	7	WhatsApp, pasarela de pagos y padrón SUNAT/RENIEC
`webhook.test.ts`	5	Firma HMAC-SHA256 válida, inválida y ausente
`products` , `clients_dashboard` , `security` , `regression`	3 c/u	Catálogo, CRM y KPIs, cabeceras Helmet, contrato de la API
`jwt.test.ts`	2	Fail-fast de secretos y rechazo de `alg: none`

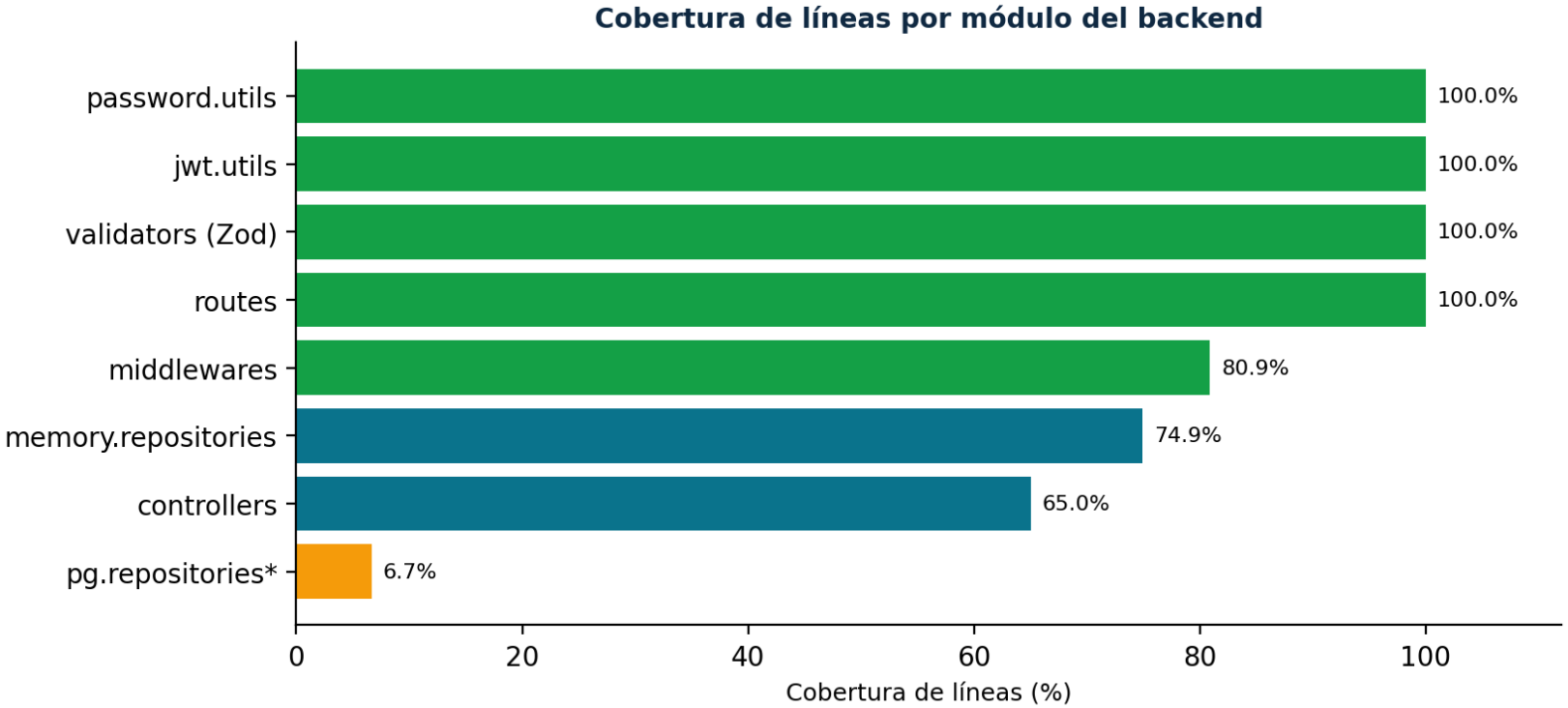
- **Frameworks:** Jest, ts-jest y Supertest en backend; Vitest y Testing Library en frontend.

Resultado Global de la Batería de Pruebas

Pruebas Aprobadas	Backend (Jest)	Frontend (Vitest)	Cobertura de Líneas
101	47	54	76.16%
0 fallos	Más 1 prueba condicionada	8 archivos de prueba	Lógica de negocio

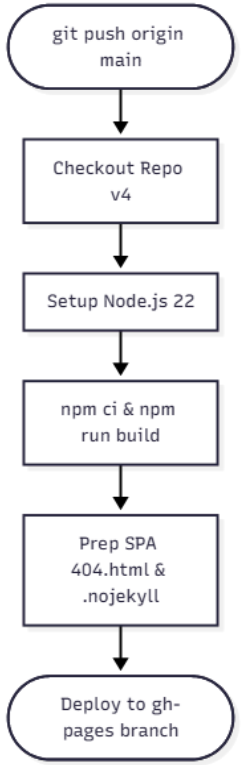
Cobertura del **100%** en rutas, validadores y criptografía.

Cobertura por Módulo



* Adaptador PostgreSQL: requiere base real (suite integration_repositories condicionada por RUN_INTEGRATION_TESTS).

Pipeline de Despliegue del Frontend (deploy.yml)



- 3 workflows: `backend-ci-cd.yml`, `deploy.yml` y `security-scan.yml`.

IV. Interoperabilidad

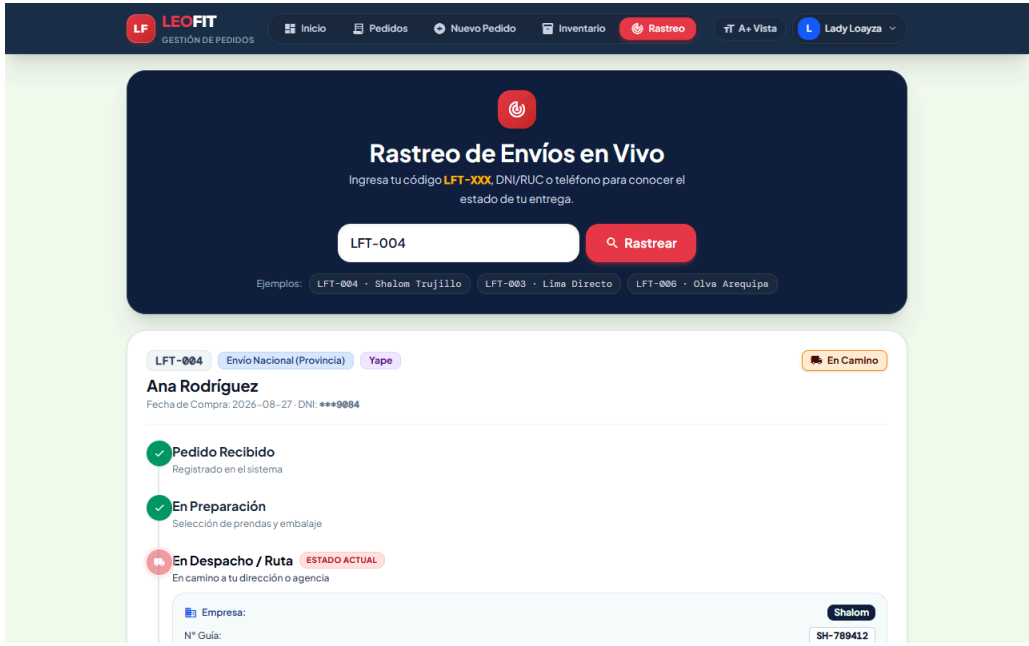
Integración con servicios externos

Expositor: Harley Anthony Roman Delgado

Integración con Sistemas Externos

- **WhatsApp:** enlace estructurado con el pedido (`#LFT-XXX`) y URL pública de seguimiento, con validación de teléfono internacional peruano.
- **Pasarelas de pago (Yape / Plin):** conciliación mediante webhooks con firma HMAC-SHA256.
- **Padrón de identidad:** validación de DNI (8 dígitos) y RUC (11 dígitos, condición HABIDO).
- **Rótulos logísticos:** PDF con código QR para despacho por agencias de transporte.

> Los proveedores se consumen mediante contratos simulados, listos para conectarse a las APIs reales.



Pruebas de Integración Externa

Código	Sistema	Escenario Evaluado	Estado
`ext-01`	WhatsApp	Notificación con URL pública de seguimiento	✅ PASSED
`ext-02`	WhatsApp	Rechazo de teléfonos sin formato internacional	✅ PASSED
`ext-03`	Pagos	Conciliación del webhook de pago aprobado	✅ PASSED
`ext-04`	Pagos	Rechazo de proveedores no homologados	✅ PASSED
`ext-05`	RENIEC	Validación de DNI de 8 dígitos	✅ PASSED
`ext-06`	SUNAT	Validación de RUC de 11 dígitos y condición HABIDO	✅ PASSED
`ext-07`	Identidad	Rechazo de documentos malformados	✅ PASSED

- **Evidencia:** `backend/tests/integration_external.test.ts`.

V. Usabilidad ISO/IEC 25010

Medición con usuarios y pruebas automatizadas

Expositor: Harley Anthony Roman Delgado

Subatributos de Usabilidad Evaluados

Subatributo ISO/IEC 25010	Enfoque en LeoFit	Mecanismo Implementado
Aprendizaje (*Learnability*)	Uso inmediato sin capacitación	Catálogo visual y flujo guiado de pedido
Protección contra errores (*Error Protection*)	Prevenir equivocaciones al registrar	Stock no negativo, validación de DNI/RUC y tope de cupones
Asistencia al usuario (*User Assistance*)	Retroalimentación clara	Código `#LFT-XXX`, resumen del pedido y seguimiento autónomo
Involucramiento (*User Engagement*)	Interfaz fluida y atractiva	Micro-interacciones y modo de alto contraste

- **Evidencia automatizada:** 9 pruebas en `frontend/src/__tests__/usability_iso25010.test.ts`.

Resultados Cuantitativos de Usabilidad

Escala SUS	Tiempo de Captura	Errores Operativos
88.5 / 100	**38.2 s**	**0.4%**
Estudio con 10 usuarios	Antes: 45 minutos	Antes: 14.2% en despachos

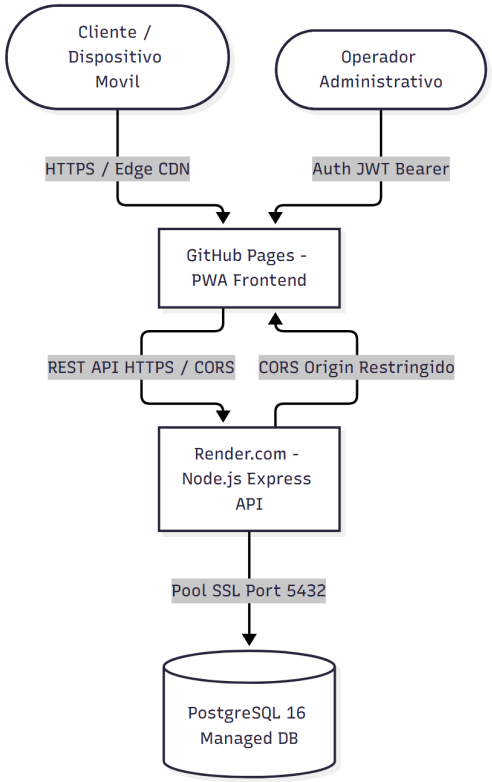
Core Web Vitals en producción: LCP **1.4 s** · INP **45 ms** · CLS **0.01** · TTFB **< 180 ms**

VI. Despliegue Cloud y Gobernanza

Versión 2 en producción

Expositor: Daniel Enrique Rojas Sanchez

Topología Cloud Desplegada



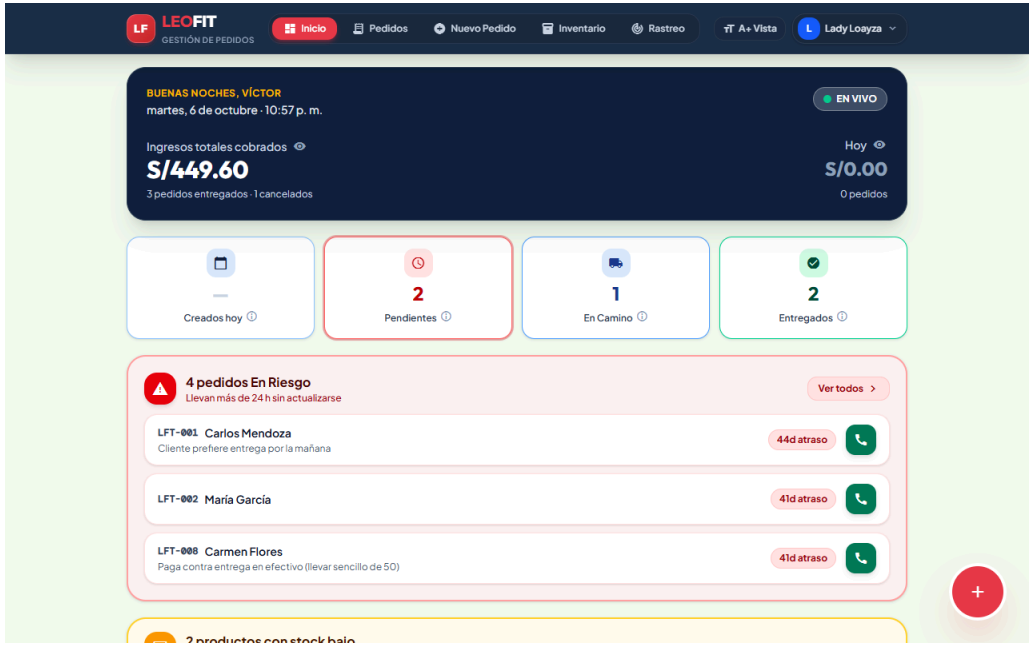
- **Frontend PWA:** GitHub Pages (CDN global con HTTPS).
- **Backend y datos:** servicio web Node 20 y PostgreSQL 16 gestionado en Render.

Validación en Vivo

URL de Producción

https://leofit-solutions-grupo01.github.io/leofit-pedidos-sistema/

- **Salud de la API:** `GET https://leofit-backend-api.onrender.com/api/health`
- **Flujo a demostrar:** login de operador, catálogo, creación de pedido y seguimiento público.



Trazabilidad y Gestión del Repositorio

Aspecto	Evidencia en GitHub	Aporte
Historial	Más de 260 commits con autoría individual	Desarrollo continuo y verificable
Ramas	`feature/<integrante>` por cada miembro y `main`	Trabajo paralelo sin colisiones
Mensajes	*Conventional Commits* (`feat:` , `fix:` , `test:` , `docs:`)	Bitácora de cambios legible
CI/CD	GitHub Actions: pruebas, despliegue y escaneo de seguridad	Verificación antes de publicar

- **Repositorio:** `github.com/Leofit-Solutions-Grupo01/leofit-pedidos-sistema`

Aportes del Equipo

Integrante	Rol	Aporte Principal
Lady Luz Loayza	Scrum Master / DevSecOps	Seguridad, gestión del sprint y consolidación documental
Víctor Cárdenas	Product Owner / DBA	Modelo de datos, transacciones y priorización del backlog
Harley Roman	Frontend Lead / PWA	PWA, interoperabilidad y usabilidad
Jim Dávila	QA Lead / Backend	Batería de 101 pruebas, cobertura y pipeline de calidad
Daniel Rojas	Analista de Negocio / Cloud DevOps	Análisis de procesos, despliegue en la nube y trazabilidad

Conclusiones y Trabajo Futuro

Logros	Limitaciones Reconocidas	Hoja de Ruta
Pedido en 38.2 s frente a 45 min	Arranque en frío en el plan gratuito de Render	Modo offline con Workbox / Service Worker
Stock consistente bajo concurrencia	Offline basado en caché, sin IndexedDB en producción	Pasarela de pago directa con cumplimiento PCI-DSS
101 pruebas aprobadas y CI/CD activo	Proveedores externos con contratos simulados	Contenedor de backend dedicado

¡Muchas Gracias!

¿Preguntas del Jurado?

Repositorio:

`github.com/Leofit-Solutions-Grupo01/leofit-pedidos-sistema`

URL de Producción:

`leofit-solutions-grupo01.github.io/leofit-pedidos-sistema`

Equipo de Ingeniería — Grupo 01 | UTP 2026